Лабораторная работа №2: Алгоритмы растеризации на Canvas

Цель работы: Приобрести навыки реализации базовых алгоритмов компьютерной графики на низком уровне. Понять, как устроены внутри графические примитивы, и научиться работать с пиксельным доступом к изображению. Осознать разницу между использованием готового API и самостоятельной реализацией алгоритмов.

Теоретическое обоснование

1. От высокоуровневого API к низкоуровневым алгоритмам

В Лабораторной работе №1 мы использовали готовые методы Canvas:

- lineTo(x, y) - рисование линии
- fillRect(x, y, w, h) - заливка прямоугольника
- fill() - заливка области

Но как они работают внутри?

Любой графический примитив в конечном итоге сводится к установке цвета конкретных пикселей на экране. Задача алгоритмов растеризации - определить, какие именно пиксели нужно закрасить для отображения заданной геометрической фигуры.

```
// ЛР 1: Как мы рисовали линию
ctx.beginPath();
ctx.moveTo(10, 10);
ctx.lineTo(100, 100);  // Магия! Браузер сам рисует линию
ctx.stroke();

// ЛР 2: Как это работает внутри
function bresenhamLine(x1, y1, x2, y2) {
    const pixels = [];
    // ... 20 строк алгоритма ...
    // Самостоятельно вычисляем, какие пиксели зажечь
    pixels.forEach(p => setPixel(p.x, p.y, color));
}
```

2. Пиксельный доступ в Canvas

Для низкоуровневой работы Canvas предоставляет ImageData:

```
// Получаем доступ к пикселям
const imageData = ctx.getImageData(0, 0, width, height);
const data = imageData.data;  // Uint8ClampedArray [R,G,B,A, R,G,B,A, ...]

// Устанавливаем пиксель (x, y) цвета (r,g,b,a)
const index = (y * width + x) * 4;
data[index] = r;      // Красный
data[index + 1] = g;  // Зеленый
data[index + 2] = b;  // Синий
data[index + 3] = a;  // Прозрачность

// Возвращаем на холст
ctx.putImageData(imageData, 0, 0);
```

Важно: Это медленнее, чем встроенные методы, но дает полный контроль над каждым пикселем!

3. Алгоритм Брезенхема (Bresenham's Line Algorithm)

Проблема: Как на дискретной сетке пикселей нарисовать непрерывную линию?

Наивное решение (неэффективное):

```
// Использует float, медленно, есть проблемы с вертикальными линиями
for(let x = x1; x <= x2; x++) {
    const y = m * x + b;
    setPixel(x, Math.round(y), color);
}
```

Алгоритм Брезенхема (1962):

- Использует только целочисленную арифметику
- Не требует умножения и деления
- Минимальное количество операций на пиксель
- Работает для всех октантов

Математическая основа:

Уравнение прямой: $y = (dy/dx)\ x + b$

Умножим на dx: $y\ dx = dy\ x + b\ dx$

Введем ошибку $\varepsilon = 2dy - dx$

Логика принятия решения:

- Если $\varepsilon > 0$: следующий пиксель по диагонали, $\varepsilon \mathrel{+}= 2dy - 2dx$
- Иначе: следующий пиксель по горизонтали, `$\varepsilon \mathrel{+}= 2dy$`

Визуализация ошибки:

```
Идеальная линия: ----.
                    \
                     \
                      '----

Пиксели:             ▙
                      ▙
                       ▙
                        ▙

Ошибка: расстояние от центра пикселя до идеальной линии
```

Ошибка: расстояние от центра пикселя до идеальной линии

4. Алгоритмы заливки (Fill Algorithms)

Проблема: Как закрасить связную область?

Два подхода:

1. Flood Fill (заливка из точки):
   - Начинаем с точки внутри области
   - Рекурсивно/итеративно распространяемся на соседей
   - Требует знания "целевого" цвета границы

2. Scanline Fill (заливка сканирующими строками):
   - Находим пересечения сканирующей строки с границами фигуры
   - Закрашиваем отрезки между пересечениями
   - Эффективнее для больших областей

Виды связности:
- 4-связность: соседи по горизонтали и вертикали
- 8-связность: + соседи по диагонали

5. Растеризация треугольника (Triangle Rasterization)

Проблема: Как определить, какие пиксели принадлежат треугольнику?

Три основных метода:

1. Bounding Box + Half-space тест:
   - Находим ограничивающий прямоугольник
   - Для каждого пикселя проверяем, внутри ли он треугольника
   - Используем барицентрические координаты

2. Scanline метод:
   - Сортируем вершины по Y
   - Для каждой сканирующей строки находим пересечения с ребрами
   - Закрашиваем отрезок между пересечениями

3. Edge Function (полуплоскости):
   - Используем знак векторного произведения
   - Точка внутри, если все три edge-функции ≥ 0

Барицентрические координаты:

Любая точка P внутри треугольника ABC может быть выражена как:

$P = \alpha \cdot A + \beta \cdot B + \gamma \cdot C$, где $\alpha + \beta + \gamma = 1$, $\alpha,\beta,\gamma \geq 0$

Интерполяция атрибутов:

$color = \alpha \cdot colorA + \beta \cdot colorB + \gamma \cdot colorC$

$uv = \alpha \cdot uvA + \beta \cdot uvB + \gamma \cdot uvC$

$normal = \alpha \cdot normalA + \beta \cdot normalB + \gamma \cdot normalC$

Практические примеры

Пример 1. Пиксельный буфер

```
class PixelBuffer {
    constructor(width, height) {
        this.width = width;
```

```
        this.height = height;
        this.canvas = document.createElement('canvas');
        this.canvas.width = width;
        this.canvas.height = height;
        this.ctx = this.canvas.getContext('2d');
        this.imageData = this.ctx.createImageData(width, height);
        this.data = this.imageData.data;
    }

    // Установка пикселя с проверкой границ
    setPixel(x, y, r, g, b, a = 255) {
        x = Math.floor(x);
        y = Math.floor(y);

        if (x < 0 || x >= this.width || y < 0 || y >= this.height) {
            return false;
        }

        const index = (y * this.width + x) * 4;
        this.data[index] = r;
        this.data[index + 1] = g;
        this.data[index + 2] = b;
        this.data[index + 3] = a;
        return true;
    }

    // Получение цвета пикселя
    getPixel(x, y) {
        x = Math.floor(x);
        y = Math.floor(y);

        if (x < 0 || x >= this.width || y < 0 || y >= this.height) {
            return null;
        }

        const index = (y * this.width + x) * 4;
        return {
            r: this.data[index],
            g: this.data[index + 1],
            b: this.data[index + 2],
            a: this.data[index + 3]
        };
    }

    // Очистка буфера
    clear(r = 0, g = 0, b = 0, a = 255) {
        for (let i = 0; i < this.data.length; i += 4) {
            this.data[i] = r;
            this.data[i + 1] = g;
            this.data[i + 2] = b;
            this.data[i + 3] = a;
```

```
        }
    }

    // Вывод на экран
    render() {
        this.ctx.putImageData(this.imageData, 0, 0);
        return this.canvas;
    }
}
```

Пример 2. Алгоритм Брезенхема для всех октантов

```
class BresenhamLine {
    static draw(x1, y1, x2, y2, pixelBuffer, color) {
        let dx = Math.abs(x2 - x1);
        let dy = Math.abs(y2 - y1);
        let sx = x1 < x2 ? 1 : -1;
        let sy = y1 < y2 ? 1 : -1;
        let err = dx - dy;

        while (true) {
            pixelBuffer.setPixel(x1, y1, ...color);

            if (x1 === x2 && y1 === y2) break;

            let e2 = 2 * err;
            if (e2 > -dy) {
                err -= dy;
                x1 += sx;
            }
            if (e2 < dx) {
                err += dx;
                y1 += sy;
            }
        }
    }

    // Рисование линии с визуализацией ошибки
    static drawWithError(x1, y1, x2, y2, pixelBuffer, color) {
        const errorPoints = [];
        let cx = x1, cy = y1;
        let dx = Math.abs(x2 - x1);
        let dy = Math.abs(y2 - y1);
        let sx = x1 < x2 ? 1 : -1;
        let sy = y1 < y2 ? 1 : -1;
        let err = dx - dy;

        // Вычисляем идеальную линию для оценки ошибки
        const m = dx !== 0 ? dy / dx : Infinity;
```

```
        const b = y1 - m * x1;

        while (true) {
            pixelBuffer.setPixel(cx, cy, ...color);

            // Вычисляем ошибку для текущего пикселя
            let idealY = m * cx + b;
            let error = Math.abs(cy - idealY);
            errorPoints.push({x: cx, y: cy, error});

            if (cx === x2 && cy === y2) break;

            let e2 = 2 * err;
            if (e2 > -dy) {
                err -= dy;
                cx += sx;
            }
            if (e2 < dx) {
                err += dx;
                cy += sy;
            }
        }

        return errorPoints;
    }
}
```

Пример 3. Flood Fill с разными типами связности

```
class FloodFill {
    // Итеративная версия (без рекурсии, использует стек)
    static fill(x, y, targetColor, fillColor, pixelBuffer, connectivity = 4) {
        // Получаем цвет в начальной точке
        const startPixel = pixelBuffer.getPixel(x, y);
        if (!startPixel) return;

        const target = [startPixel.r, startPixel.g, startPixel.b];
        const fill = [...fillColor];

        // Проверяем, не совпадает ли целевой цвет с цветом заливки
        if (target[0] === fill[0] && target[1] === fill[1] && target[2] ===
fill[2]) {
            return;
        }

        const stack = [[x, y]];
        const neighbors = connectivity === 4
            ? [[0,1], [1,0], [0,-1], [-1,0]]
            : [[0,1], [1,0], [0,-1], [-1,0], [1,1], [1,-1], [-1,1], [-1,-1]];
```

```
        while (stack.length > 0) {
            const [cx, cy] = stack.pop();

            const pixel = pixelBuffer.getPixel(cx, cy);
            if (!pixel) continue;

            if (pixel.r === target[0] && pixel.g === target[1] && pixel.b ===
target[2]) {
                pixelBuffer.setPixel(cx, cy, ...fill);

                for (const [dx, dy] of neighbors) {
                    stack.push([cx + dx, cy + dy]);
                }
            }
        }
    }

    // Scanline fill (оптимизированная версия)
    static scanlineFill(x, y, targetColor, fillColor, pixelBuffer) {
        const startPixel = pixelBuffer.getPixel(x, y);
        if (!startPixel) return;

        const target = [startPixel.r, startPixel.g, startPixel.b];
        const fill = [...fillColor];

        if (target[0] === fill[0] && target[1] === fill[1] && target[2] ===
fill[2]) {
            return;
        }

        const stack = [[x, y]];

        while (stack.length > 0) {
            let [cx, cy] = stack.pop();

            // Ищем левую границу
            let left = cx;
            while (true) {
                const pixel = pixelBuffer.getPixel(left - 1, cy);
                if (!pixel || pixel.r !== target[0] || pixel.g !== target[1] ||
pixel.b !== target[2]) break;
                left--;
            }

            // Ищем правую границу
            let right = cx;
            while (true) {
                const pixel = pixelBuffer.getPixel(right + 1, cy);
                if (!pixel || pixel.r !== target[0] || pixel.g !== target[1] ||
pixel.b !== target[2]) break;
```

```
                right++;
            }

            // Заливаем строку
            for (let ix = left; ix <= right; ix++) {
                pixelBuffer.setPixel(ix, cy, ...fill);
            }

            // Проверяем строку выше
            if (cy > 0) {
                let spanAdded = false;
                for (let ix = left; ix <= right; ix++) {
                    const pixel = pixelBuffer.getPixel(ix, cy - 1);
                    if (pixel && pixel.r === target[0] && pixel.g === target[1]
&& pixel.b === target[2]) {
                        if (!spanAdded) {
                            stack.push([ix, cy - 1]);
                            spanAdded = true;
                        }
                    } else {
                        spanAdded = false;
                    }
                }
            }

            // Проверяем строку ниже
            if (cy < pixelBuffer.height - 1) {
                let spanAdded = false;
                for (let ix = left; ix <= right; ix++) {
                    const pixel = pixelBuffer.getPixel(ix, cy + 1);
                    if (pixel && pixel.r === target[0] && pixel.g === target[1]
&& pixel.b === target[2]) {
                        if (!spanAdded) {
                            stack.push([ix, cy + 1]);
                            spanAdded = true;
                        }
                    } else {
                        spanAdded = false;
                    }
                }
            }
        }
    }
}
```

Пример 4. Растеризация треугольника с интерполяцией цвета

```
class TriangleRasterizer {
    // Edge function (полуплоскость)
```

```
    static edgeFunction(a, b, c) {
        return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
    }

    // Барицентрические координаты
    static barycentric(p, a, b, c) {
        const area = this.edgeFunction(a, b, c);

        // Если треугольник вырожденный
        if (Math.abs(area) < 1e-6) return null;

        const w0 = this.edgeFunction(b, c, p);
        const w1 = this.edgeFunction(c, a, p);
        const w2 = this.edgeFunction(a, b, p);

        return [w0 / area, w1 / area, w2 / area];
    }

    // Растеризация с интерполяцией цвета (метод bounding box)
    static draw(v1, v2, v3, pixelBuffer) {
        // Находим bounding box
        const minX = Math.max(0, Math.floor(Math.min(v1.x, v2.x, v3.x)));
        const maxX = Math.min(pixelBuffer.width - 1, Math.ceil(Math.max(v1.x,
v2.x, v3.x)));
        const minY = Math.max(0, Math.floor(Math.min(v1.y, v2.y, v3.y)));
        const maxY = Math.min(pixelBuffer.height - 1, Math.ceil(Math.max(v1.y,
v2.y, v3.y)));

        // Предвычисляем константы для оптимизации
        const area = this.edgeFunction(v1, v2, v3);

        for (let y = minY; y <= maxY; y++) {
            for (let x = minX; x <= maxX; x++) {
                const p = {x, y};

                // Вычисляем барицентрические координаты
                const w0 = this.edgeFunction(v2, v3, p) / area;
                const w1 = this.edgeFunction(v3, v1, p) / area;
                const w2 = this.edgeFunction(v1, v2, p) / area;

                // Проверяем, внутри ли пиксель треугольника
                if (w0 >= 0 && w1 >= 0 && w2 >= 0) {
                    // Интерполяция цвета
                    const r = w0 * v1.color[0] + w1 * v2.color[0] + w2 *
v3.color[0];
                    const g = w0 * v1.color[1] + w1 * v2.color[1] + w2 *
v3.color[1];
                    const b = w0 * v1.color[2] + w1 * v2.color[2] + w2 *
v3.color[2];

                    pixelBuffer.setPixel(x, y, r, g, b, 255);
```

```
                }
            }
        }
    }

    // Растеризация с использованием scanline (более эффективная)
    static drawScanline(v1, v2, v3, pixelBuffer) {
        // Сортируем вершины по Y
        const vertices = [v1, v2, v3].sort((a, b) => a.y - b.y);
        const [top, middle, bottom] = vertices;

        // Вычисляем градиенты
        const totalHeight = bottom.y - top.y;

        // Верхняя часть треугольника (top -> middle)
        for (let y = Math.floor(top.y); y <= Math.floor(middle.y); y++) {
            if (y < 0 || y >= pixelBuffer.height) continue;

            const alpha = (y - top.y) / totalHeight;
            const beta = (y - top.y) / (middle.y - top.y || 1);

            // Вычисляем X-координаты пересечений
            const x1 = top.x + (bottom.x - top.x) * alpha;
            const x2 = top.x + (middle.x - top.x) * beta;

            const startX = Math.floor(Math.min(x1, x2));
            const endX = Math.ceil(Math.max(x1, x2));

            // Интерполяция цвета вдоль строки
            for (let x = startX; x <= endX; x++) {
                if (x < 0 || x >= pixelBuffer.width) continue;

                const t = (x - startX) / (endX - startX || 1);
                const r = this.lerp(
                    this.lerp(top.color[0], bottom.color[0], alpha),
                    this.lerp(top.color[0], middle.color[0], beta),
                    t
                );
                const g = this.lerp(
                    this.lerp(top.color[1], bottom.color[1], alpha),
                    this.lerp(top.color[1], middle.color[1], beta),
                    t
                );
                const b = this.lerp(
                    this.lerp(top.color[2], bottom.color[2], alpha),
                    this.lerp(top.color[2], middle.color[2], beta),
                    t
                );

                pixelBuffer.setPixel(x, y, r, g, b, 255);
            }
```

```
        }

        // Нижняя часть треугольника (middle -> bottom)
        for (let y = Math.floor(middle.y); y <= Math.floor(bottom.y); y++) {
            if (y < 0 || y >= pixelBuffer.height) continue;

            const alpha = (y - top.y) / totalHeight;
            const beta = (y - middle.y) / (bottom.y - middle.y || 1);

            const x1 = top.x + (bottom.x - top.x) * alpha;
            const x2 = middle.x + (bottom.x - middle.x) * beta;

            const startX = Math.floor(Math.min(x1, x2));
            const endX = Math.ceil(Math.max(x1, x2));

            for (let x = startX; x <= endX; x++) {
                if (x < 0 || x >= pixelBuffer.width) continue;

                const t = (x - startX) / (endX - startX || 1);
                const r = this.lerp(
                    this.lerp(top.color[0], bottom.color[0], alpha),
                    this.lerp(middle.color[0], bottom.color[0], beta),
                    t
                );
                const g = this.lerp(
                    this.lerp(top.color[1], bottom.color[1], alpha),
                    this.lerp(middle.color[1], bottom.color[1], beta),
                    t
                );
                const b = this.lerp(
                    this.lerp(top.color[2], bottom.color[2], alpha),
                    this.lerp(middle.color[2], bottom.color[2], beta),
                    t
                );

                pixelBuffer.setPixel(x, y, r, g, b, 255);
            }
        }
    }

    static lerp(a, b, t) {
        return a + (b - a) * t;
    }
}
```

Пример 5. Суперсэмплинг для антиалиасинга

```
class SupersampledBuffer {
    constructor(width, height, factor = 2) {
```

```
        this.width = width;
        this.height = height;
        this.factor = factor;
        this.buffer = new PixelBuffer(width * factor, height * factor);
    }

    // Рисуем в буфер повышенного разрешения
    getPixelBuffer() {
        return this.buffer;
    }

    // Понижающая дискретизация (downsampling)
    resolve() {
        const result = new PixelBuffer(this.width, this.height);

        for (let y = 0; y < this.height; y++) {
            for (let x = 0; x < this.width; x++) {
                let r = 0, g = 0, b = 0, count = 0;

                // Усредняем все субпиксели
                for (let dy = 0; dy < this.factor; dy++) {
                    for (let dx = 0; dx < this.factor; dx++) {
                        const sx = x * this.factor + dx;
                        const sy = y * this.factor + dy;
                        const pixel = this.buffer.getPixel(sx, sy);

                        if (pixel) {
                            r += pixel.r;
                            g += pixel.g;
                            b += pixel.b;
                            count++;
                        }
                    }
                }

                if (count > 0) {
                    result.setPixel(x, y,
                        Math.round(r / count),
                        Math.round(g / count),
                        Math.round(b / count),
                        255
                    );
                }
            }
        }

        return result;
    }
}
```

Задания для самостоятельного выполнения

Базовый уровень (обязательно для всех)

Задание 1. Реализация алгоритма Брезенхема

Реализуйте функцию рисования линии по алгоритму Брезенхема:

- Поддержка всех 8 октантов (любые направления)
- Возможность задавать цвет линии
- Возможность задавать толщину линии (опционально)
- Визуализация ошибки аппроксимации (разными цветами)

Сравните с встроенным lineTo():

- Замерьте производительность (10000 линий)
- Оцените визуальное качество
- Сделайте выводы

Задание 2. Реализация Flood Fill

Реализуйте алгоритм заливки области:

- Поддержка 4-связности и 8-связности
- Итеративная реализация (стек/очередь)
- Оптимизированная версия (scanline fill)
- Возможность заливки с допуском (tolerance) для похожих цветов

Сравните производительность разных подходов на областях 100×100

Задание 3. Растеризация треугольника

Реализуйте растеризатор треугольника с интерполяцией цвета:

- Метод bounding box + барицентрические координаты
- Метод scanline (для сравнения производительности)
- Правильная интерполяция цвета между вершинами

- Back-face culling (отсечение нелицевых граней)

Создайте сцену из 100 треугольников с градиентной заливкой

Задание 4. Рисование окружности

Реализуйте алгоритм Брезенхема для окружности:

- Рисование окружности произвольного радиуса
- Только целочисленная арифметика
- Заливка круга (растровая развертка)
- Антиалиасинг (алгоритм Ву для окружностей)

Сравните с встроенным arc() по качеству и скорости

Задание 5. Простой Z-буфер

Реализуйте Z-буфер для правильного отображения перекрытий:

- Массив глубины для каждого пикселя
- Сортировка треугольников по глубине
- Рендеринг нескольких перекрывающихся треугольников
- Визуализация глубины (чем ближе, тем ярче)

Создайте сцену из 3 пересекающихся треугольников

Задание 6. Антиалиасинг суперсэмплингом

Реализуйте SSAA 2x, 4x:

- Рендеринг в буфер повышенного разрешения
- Усреднение субпикселей для финального изображения
- Адаптивный суперсэмплинг (больше выборок на границах)
- Сравнение качества/производительности с MSAA

Нарисуйте тонкую диагональную линию (45°) и сравните с обычной

Задание 7. Градиентная заливка многоугольника

Реализуйте заливку произвольного выпуклого многоугольника:

- Интерполяция цвета по градиенту
- Поддержка N-угольников ($N \geq 3$)
- Правильный порядок вершин (CW/CCW)
- Отсечение по границам экрана (clipping)

Создайте шестиугольник с радиальным градиентом

Средний уровень

Задание 1. Алгоритм Ву для сглаженных линий

Реализуйте алгоритм Xiaolin Wu для антиалиасинга линий:

- Сглаживание "ступенек" через альфа-канал
- Поддержка произвольной толщины линии
- Корректная работа с полупрозрачностью
- Сравнение с Брезенхемом и встроенным lineTo()

Визуализируйте разницу в качестве (зум 400%)

Задание 2. Текстурный треугольник

Реализуйте текстурированный треугольник:

- UV-координаты для вершин
- Перспективно-корректная интерполяция текстур
- Билинейная фильтрация текстур
- Мип-маппинг (упрощенный)

Загрузите текстуру и наложите на вращающийся треугольник

Задание 3: Отсечение отрезков (Cohen-Sutherland)

Реализуйте алгоритм отсечения отрезков:

- Коды регионов (outcodes) для 9 зон
- Эффективное отбрасывание невидимых отрезков
- Отсечение по прямоугольному окну
- Визуализация процесса отсечения

Создайте интерактивную демонстрацию с изменяемым окном

Задание 4. Растеризация кривых Безье

Реализуйте растеризацию квадратичных и кубических кривых Безье:

- Рекурсивное подразделение (de Casteljau)
- Адаптивная тесселяция (по кривизне)
- Рисование с контролем ошибки
- Интерактивное перемещение контрольных точек

Создайте редактор кривых с 4+ контрольными точками

Задание 5. Процедурная генерация шумов

Реализуйте генерацию шума Перлина (1D/2D):

- Интерполяция градиентов
- Мультиоктавный шум (фрактальный)
- Визуализация как heightmap
- Процедурная генерация текстур (дерево, мрамор)

Создайте ландшафт с освещением по высоте

Задание 6: Морфинг фигур

Реализуйте плавную трансформацию одной фигуры в другую:

- Соответствие вершин (интерполяция)
- Анимация во времени
- Несколько ключевых кадров
- Сглаживание переходов

Пример: квадрат → круг, треугольник → звезда

Задание 7. Dithering для ограниченной палитры

Реализуйте алгоритмы дизеринга:

- Ordered dither (матрицы Байера)
- Floyd-Steinberg (распространение ошибки)
- Адаптация к индексированным цветам
- Сравнение качества на градиентах

Преобразуйте фотографию в 16 цветов с дизерингом

Продвинутый уровень

Задание 1. Программный 3D-движок

Реализуйте полноценный 3D-рендерер на 2D Canvas:

- Матричные преобразования (MVP)
- Перспективная проекция
- Z-буфер для глубины
- Модель освещения Фонга (ambient + diffuse + specular)
- Загрузка простых .obj моделей
- Управление камерой (WASD + мышь)

Результат: вращающийся куб/сфера с освещением и тенями

Задание 2. Ray marching в шейдере

Реализуйте ray marching для SDF (signed distance fields):

- Базовые примитивы (сфера, куб, тор)
- CSG операции (объединение, вычитание, пересечение)
- Мягкие тени и ambient occlusion
- Материалы и освещение
- Анимация сцены во времени

Создайте сцену из 5+ примитивов с отражениями

Задание 3: Симуляция частиц с физикой

Реализуйте систему частиц (10k+ частиц):

- Векторное поле сил (гравитация, ветер)

Контрольные вопросы к защите лабораторной работы

1. Опишите алгоритм Брезенхема для рисования отрезка. Как принимается решение о следующем пикселе и почему алгоритм использует только целочисленные операции?
2. В чём принципиальная разница между алгоритмами Flood Fill и Scanline Fill? Какой из них эффективнее для больших областей и почему?
3. Что такое барицентрические координаты? Как с их помощью определить, находится ли точка внутри треугольника, и выполнить интерполяцию цвета?
4. Объясните принцип работы Z-буфера. Какая проблема компьютерной графики решается с его помощью и какие артефакты могут возникнуть при его использовании?
5. Как работает метод суперсэмплинга (SSAA) для сглаживания ступенчатых краёв? Почему SSAA 4x требует в 4 раза больше памяти и в ~4 раза медленнее обычного рендеринга?

6. Как выполнить градиентную заливку выпуклого многоугольника? Почему для выпуклых многоугольников задача проще, чем для невыпуклых?
7. Как модифицировать алгоритм Брезенхема для рисования окружности? Какая симметрия используется для оптимизации?
8. Как математически описать простую одноточечную перспективу? Почему шахматная доска в перспективе требует неравномерного деления клеток?
9. Чем ваша программная реализация растеризации отличается от того, как это делает GPU? Почему ваша реализация медленнее в сотни раз?
10. В Лабораторной работе №1 вы использовали готовые методы Canvas (lineTo, fill). В Лабораторной работе №2 вы реализовали эти же алгоритмы сами. Что вы поняли о том, как работает «чёрный ящик» браузера? Какие ограничения вашей реализации вы увидели?